Lab 6-1
SystemVerilog Architecture

Prof. Chih-Cheng Tseng (曾志成)

cctseng@mail.ntou.edu.tw

Contents of this slide are provided by NTOU VLSI Lab601

SystemVerilog Architecture

- The main structure of SystemVerilog is the module.
  - A module is like a building block
  - Each large digital system is made up of building blocks with specific functions.
  - A project consists of multiple modules.
  - Each SystemVerilog file must contain one module.
- Structure of a module:

```
module module_name (input/output port names and description);
    input/output port description;
    data type description;
    internal circuit description;
endmodule
```

- The module name and input/output port names are responsible for providing descriptive functionality. Avoid arbitrary naming.
- SystemVerilog uses two types of comment syntax: // and /**/.

Input and Output Port Names and Description

- Example:

```
module module_name (
        input in1, in2,
        output out1, out2,
        inout inout1
);
```

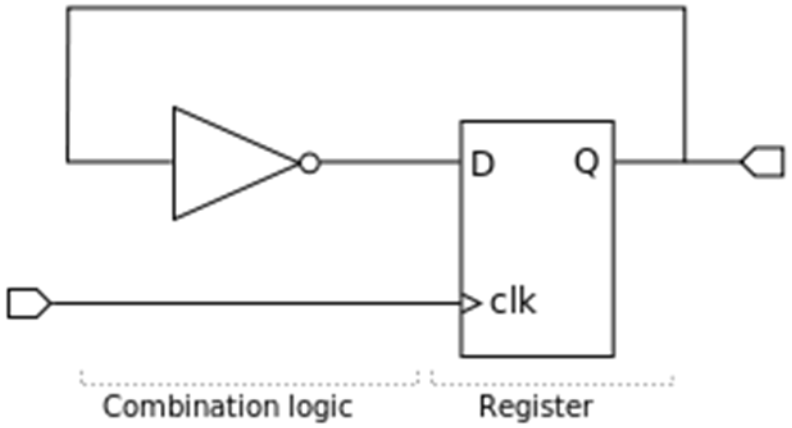
  - The input and output order can be decided independently.
  - Port names are separated by commas
  - No symbol is added after the last port name.

Data Type Descriptions

- When using Verilog, **wire** and **reg** were commonly used
- In SystemVerilog, **logic** can be used instead. The compiler will automatically determine whether it is one of the two types mentioned above.
- The most important part of SystemVerilog is to describe the module's circuit structure and functionality.
- There are four primary levels of description (from high-level to low-level):
  - Behavior Level
  - Dataflow Level
  - Gate Level
  - Switch Level
- The behavior and dataflow levels are collectively known as the Register Transfer Level (RTL).
- When writing SystemVerilog, the switch level will not be encountered.

RTL (Register Transfer Level)

- A synchronous circuit is composed of two main elements
  - Combinational logic circuits
  - Registers
    - Usually made up of D flip-flops
    - Perform synchronized timing operations based on a given clock pulse.
    - Provide sequential logic circuits the ability to store memory.



Examples of The Descriptions in Different Levels

- Descriptions of performing an AND operation between In1 and In2:
  - Gate level:

```
and and1( Out1, In1, In2 );
// LogicGate name (output, input, input)
```

  - Dataflow level:

```
assign Out1 = In1 & In2;
// The circuit described after assign will change the output immediately
when the input changes. (Used in combinational logic circuits)
```

  - Behavior level:

```
always_comb begin
    Out2 = In1 & In2;
end
// always_comb does not require @ after it for combinational logic.
// always_ff@(posedge clk) is used for edge-triggered sequential logic.
```